PYTHON

What is Python

Python is a general-purpose, high-level, and interpreted programming language used for web development, application development, automation, and data analysis. It is easy to learn, beginner-friendly, and supports object-oriented programming. Due to its simple syntax and powerful features, Python is widely used for rapid application development.

Why Python is poplular

Python is one of the most popular programming languages because of the following reasons:

1.**Easy to Learn**

Simple and readable syntax

Beginner-friendly

2.**Versatile**

Used for web development, data science, AI, machine learning, automation, cybersecurity, and more

3.**Large Community Support**

Huge global community

Many tutorials and resources available

4.**Rich Libraries & Frameworks**

NumPy, Pandas (Data Science)

TensorFlow, PyTorch (AI/ML)

Django, Flask (Web Development)

5.**Cross-Platform**

Works on Windows, Linux, macOS

6.**High Demand in Industry**

Widely used by companies for software development and AI projects

What Can Be Done Using Python?

Python is widely used for:

- **Web and Software Development** – Building websites and application backends using frameworks like Django and Flask.
- **Data Analysis & Machine Learning** – Performing statistical analysis, data visualization, and developing machine learning models.
- **Automation & Scripting** – Automating repetitive tasks such as file handling, data processing, and error checking.
- **Data Visualization** – Creating charts and graphical reports.

Python is popular among developers, scientists, and professionals due to its simplicity and versatility.

Why learn Python?

Python provides many useful features that make it a popular and widely used programming language. Some key features are:

- **Interpreted Language** – Executes code line by line without compilation.
- **Object-Oriented Language** – Supports OOP concepts like classes and objects.
- **Open Source** – Free to use and modify.
- **Extensible** – Can be integrated with other languages like C and C++.
- **Large Standard Library** – Provides built-in modules and functions.
- **GUI Programming Support** – Supports GUI development using libraries like Tkinter.
- **Integrated** – Can work with other tools and technologies easily.
- **Embeddable** – Can be embedded into other applications.
- **Dynamic Memory Allocation** – Automatically manages memory.

Modules in Python

A **module** is a file that contains Python code (functions, variables, classes) which can be reused in other programs.

Types of Modules:

- **Built-in Modules** → Example: math, random, datetime
- **User-defined Modules** → Created by the user
- **External Modules** → Installed using pip (e.g., numpy, pandas)

Importing a Module:

```
import math
print(math.sqrt(25))
```

Import Specific Function:

```
from math import sqrt
print(sqrt(16))
```

Comments in Python

A **comment** is text in a program that explains the code.
Comments are ignored by the Python interpreter and are not executed.

1.Single-Line Comment

A single-line comment is used to write a short explanation in one line.

Syntax:

```
# This is a single-line comment
```

Example

```
# This program prints Hello

print("Hello")
```

2.Multi-Line Comment

A **multi-line comment** is used to write explanations that span multiple lines.

Syntax:

```
"""

This is a

multi-line comment

"""
```

Example:-

```
"""

This program

adds two numbers

"""
```

```
a = 5

b = 10

print(a + b)
```

PIP in Python

PIP (Preferred Installer Program) **is a package management tool used to** install, update, and manage external Python libraries and packages**.**

Why PIP is Used

Python has many additional libraries (like NumPy, Pandas, Flask).
PIP helps us install and manage these libraries easily.

Common PIP Commands

- Check PIP Version

  ```
  pip –version
  ```

- Install a Package

  ```
  pip install numpy
  ```

- Uninstall a Package

  ```
  pip uninstall numpy
  ```

- List Installed Packages

  ```
  pip list
  ```

- Upgrade a Package

  ```
  pip install --upgrade numpy
  ```

Variables

A **variable** is a name used to store data or a value in a program.

Python does not require explicit declaration of variables.

Example:

```
x = 10

name = "Anamika"
```

Here:

- x stores an integer value
- name stores a string value

Rules for Naming Variables:

- Must start with a letter or underscore (_)
- Cannot start with a number
- Cannot use keywords
- Case-sensitive (Name ≠ name)

Identifiers

An **identifier** is the name given to a variable, function, class, module, or any other object in Python.

Identifiers are used to identify and access these elements in a program.

Example:-

```
x = 10          # x is an identifier (variable)

name = "Anu"    # name is an identifier

def add():      # add is an identifier (function)

    pass
```

Data Types

A data type specifies the type of value a variable can store.

Example:-

```
a = 5        # int

b = 2.5      # float

c = "Hello"  # string
```

d = True # Boolean

Keywords

Keywords are reserved words in Python that have special meanings and cannot be used as variable names.

Examples of Python Keywords:

```
if, else, elif, for, while, break, continue,
True, False, None, and, or, not,
def, return, class, import, from, pass, try, except
```

Example:-

```
if x > 5:

    print("Greater") # if is a keyword
```

Operators in Python

Operators are special symbols used to perform operations on variables and values.

Types of Operators in Python

1.Arithmetic operators -Used to perform mathematical operations.

Operator	Meaning	Example
+	Addition	`5 + 3` → 8
-	Subtraction	`5 - 3` → 2
*	Multiplication	`5 * 3` → 15
/	Division	`5 / 2` → 2.5
%	Modulus (remainder)	`5 % 2` → 1
//	Floor Division	`5 // 2` → 2
**	Exponent	`2 ** 3` → 8

2.Comparison operators- Used to compare values.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than

Operator	Meaning
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example

```
x = 5

print(x > 3)  # True
```

3.Assignment Operators- Used to combine conditions.

Operator	Example
=	**x = 5**
+=	**x += 3**
-=	**x -= 2**
*=	**x *= 2**
/=	**x /= 2**

4.Logical Operators- Used to assign values.

Operator	Meaning
and	True if both conditions are true
or	True if at least one condition is true
not	Reverses the result

Example:-

```
x = 5

print(x > 3 and x < 10)
```

5.Membership Operators- Used to check membership in a sequence.

`in`, `not in`

Example:-

```
a = [1, 2, 3]

print(2 in a)  # True
```

6.Identity Operators- Used to compare memory location.

`is`, `is not`

Example:-

```
x = 5
```

```
y = 5
print(x is y)
```

Input Function

The **input() function** in Python is used to take input from the user during program execution.
It reads the input as a **string** by default.

Syntax:-

```
input("Prompt message")
```

Example:-

```
name = input("Enter your name: ")

print("Hello", name)
```

Parameter

Parameters are variables listed inside the parentheses of a function definition.
They are used to receive values when the function is called.

Argument

An **argument** is the actual value that is passed to a function when it is called.

Parameters are defined in the function,
Arguments are the values given to those parameters.

Example:-

```
def greet(name):   # 'name' is a parameter
    print("Hello", name)
greet("Anamika")   # "Anamika" is an argument
```

Types of Arguments in Python

1.Positional Arguments- Passed in the correct order.

```
Eg) def add(a, b):
        print(a + b)
```

```
add(5, 3)
```

2.Keyword Arguments- Passed using parameter names.

Eg)
```
def student(name, age):
    print(name, age)
student(age=20, name="Anu")
```

3.Default Arguments-If no value is given, default value is used.

Eg)
```
def greet(name="Guest"):
    print("Hello", name)
greet()
```

4. Variable-Length Arguments

- *args → Multiple positional arguments
- **kwargs → Multiple keyword arguments

Eg)
```
def total(*numbers):
    print(sum(numbers))
total(1, 2, 3, 4)
```

List, Tuple, Set, Dictionary in Python

These are built-in **collection data types** in Python used to store multiple values.

List

A **list** is an ordered and changeable (mutable) collection of elements.

Features:

- Allows duplicate values
- Indexed
- Can store different data types

Example:

```
my_list = [1, 2, 3, "Python"]

print(my_list[0])
```

Tuple

A **tuple** is an ordered but unchangeable (immutable) collection of elements.

Features:

- Allows duplicate values
- Indexed
- Cannot modify after creation

Example:

```
my_tuple = (1, 2, 3, "Python")

print(my_tuple[1])
```

Set

A **set** is an unordered collection of unique elements.

Features:

- No duplicate values
- Not indexed
- Mutable

Example:

```
my_set = {1, 2, 3, 3}

print(my_set)   # Output: {1, 2, 3}
```

Dictionary

A **dictionary** is a collection of key-value pairs.

Features:

- Keys must be unique
- Values can be duplicated

- Mutable
- Indexed using keys

Example:

```
my_dict = {"name": "Ana", "age": 21}

print(my_dict["name"])
```

Python Type Casting

Type Casting in Python is the process of converting one data type into another data type.

Types of Type Casting

1.Implicit Type Casting

Automatically done by Python.

Python converts smaller data type into larger data type.

Example:

```
a = 5       # int

b = 2.5     # float

c = a + b   # int converted to float automatically

print(c)    # 7.5
```

2.Explicit Type Casting

Done manually by the programmer using built-in functions.

Common Type Casting Functions:

Function	Converts To
int()	Integer
float()	Float
str()	String
list()	List
tuple()	Tuple
set()	Set

Example:-

Convert String to Integer

```
x = int("10")

print(x)
```

Flow Control in Python

Flow control refers to the order in which statements are executed in a program.
It controls the decision-making and looping process in Python.

Types of Flow Control

1.Conditional Statements (Decision Making)

Used to execute code based on conditions.

- **if Statement**
  An if statement is a conditional statement used to execute a block of code only if a specified condition is true.
  It is used for decision making in a program.

  SYNTAX

  ```
  if condition:
  statement
  ```

  EXAMPLE

  ```
  x = 10
  if x > 5:
  print("Greater than 5")
  ```

- **if-else Statement**
  An **if-else statement** is a conditional statement used to execute one block of code if the condition is **true**, and another block of code if the condition is **false**.

SYNTAX

```
if condition:
   statement1
else:
   statement2
```

EXAMPLE

```
if x % 2 == 0:
print("Even")
else:
 print("Odd")
```

- **if-elif-else Statement**

  The **if-elif-else statement** is used to check multiple conditions in a program.
  If the first condition is false, Python checks the next condition using elif.
  If none of the conditions are true, the else block executes.

  SYNTAX

```
if condition1:
   statement1
elif condition2:
   statement2
elif condition3:
   statement3
else:
   statement4
```

EXAMPLE

```
marks = 75
if marks >= 90:
   print("A")
elif marks >= 70:
   print("B")
else:
   print("C")
```

- **Nested If Statement in Python**

  A **Nested If statement** means placing an if statement **inside another if statement**.
  It is used when we need to check a condition inside another condition.

  SYNTAX

```
if condition1:

  if condition2:

    statement
```

  EXAMPLE

```
age = 20

citizen = True

if age >= 18:

  if citizen:

  print("Eligible to vote")
```

2.Looping Statements (Iteration)

Used to repeat a block of code.

- **for Loop**

A **for loop** is used to repeat a block of code multiple times.
It is mainly used to iterate over a sequence like a list, tuple, string, or range.

SYNTAX

```
for variable in sequence:
statement
```

EXAMPLE

```
for i in range(5):
    print(i)
```

- **While Loop**
  A **while loop** is used to repeat a block of code **as long as a condition is true**.
  It is mainly used when the number of iterations is not known in advance.

  SYNTAX

```
while condition:
statement
```

  EXAMPLE

```
x = 1
while x <= 5:
 print(x)
 x += 1
```

3. Control Statements

Used to change the normal flow of loops.

- **break → Stops the loop**

```
for i in range(5):
  if i == 3:
    break
  print(i)
```

- **continue → Skips current iteration**

```
for i in range(5):
```

```
    if i == 3:
        continue
    print(i)
```

- **pass → Does nothing (placeholder)**

```
if x > 5:
    pass
```